

NestJS kézikönyv (magyar)

Ez a dokumentum átfogó, magyar nyelvű útmutató a NestJS fő témaköreiről: Overview, Fundamentals, Techniques, Security, GraphQL, Websockets és Microservices. Tartalmaz javasolt példákat és kódrészleteket. A fájl UTF-8 kódolású.

Tartalomjegyzék

1. Overview — Áttekintés
2. Fundamentals — Alapok
3. Techniques — Technikák
4. Security — Biztonság
5. GraphQL
6. Websockets
7. Microservices — Mikroszolgáltatások
8. Kiegészítők: Docker, Swagger, Tesztek

1) Overview — Áttekintés

Mi a NestJS?

- A NestJS egy progresszív, Node.js alapú serveroldali keretrendszer, amely TypeScript-re épülve segíti a skálázható, karbantartható alkalmazások fejlesztését.
- Célja, hogy vállalati szintű felépítést és szervezést adjon a Node-alkalmazásokhoz, miközben használhatod az ismert JavaScript/TypeScript ökoszisztéma eszközeit (Express vagy Fastify, ORM-ek, WebSocket-könyvtárak, stb.).
- Inspirációt merít az Angular szerkezetéből (modulok, injektálható szolgáltatások, dekorátor-alapú deklaráció).

Főbb gondolatok

- Modularitás: a kódot modulokra bontva könnyebb tesztelni és újrafelhasználni.
- Dependency injection (DI): beépített IoC konténer a laza csatolásért és egyszerű tesztelhetőségért.
- Dekorátorok használata: TypeScript-dekorátorokkal (pl. @Controller, @Module, @Injectable) deklaratív felépítés.
- Platform-agnosztikus: alapértelmezett HTTP server Express, de Fastify is támogatott; lehetőség más platformokra való futtatásra is.

Gyors kezdés (CLI)

```
npm i -g @nestjs/cli
nest new project-name
```

```
cd project-name
npm run start:dev
```

2) Fundamentals — Alapok

Alapfogalmak és architektúra

- Module (@Module): a Nest alkalmazás szervező egysége. Minden modul deklarálja a controllereket, providereket (szolgáltatásokat) és exportált elemeket.
- Controller (@Controller): kezeli a bejövő kéréseket és válaszokat — tipikusan HTTP útvonalakhoz kötődik.
- Provider / Service (@Injectable): üzleti logika, adatkezelés; injektálható komponensek.
- Dependency Injection: konstruktor-injektálással történik, a konténer gondoskodik az életciklus-kezelésről.

Egyszerű alkalmazás példa

```
// src/app.module.ts
import { Module } from '@nestjs/common';
import { CatsModule } from './cats/cats.module';

@Module({
  imports: [CatsModule],
})
export class AppModule {}

// src/cats/cats.module.ts
import { Module } from '@nestjs/common';
import { CatsService } from './cats.service';
import { CatsController } from './cats.controller';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
  exports: [CatsService],
})
export class CatsModule {}

// src/cats/cats.service.ts
import { Injectable } from '@nestjs/common';

export type Cat = { id: number; name: string; age?: number };

@Injectable()
export class CatsService {
```

```

    private readonly cats: Cat[] = [{ id: 1, name: 'Mici' }];

    findAll(): Cat[] {
        return this.cats;
    }

    create(cat: Cat) {
        this.cats.push(cat);
        return cat;
    }
}

// src/cats/cats.controller.ts
import { Controller, Get, Post, Body } from '@nestjs/common';
import { CatsService, Cat } from './cats.service';

@Controller('cats')
export class CatsController {
    constructor(private readonly catsService: CatsService) {}

    @Get()
    getAll() {
        return this.catsService.findAll();
    }

    @Post()
    create(@Body() cat: Cat) {
        return this.catsService.create(cat);
    }
}

```

Middleware

```

// src/logger.middleware.ts
import { Injectable, NestMiddleware } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class LoggerMiddleware implements NestMiddleware {
    use(req: Request, res: Response, next: NextFunction) {
        console.log(`${req.method} ${req.originalUrl}`);
        next();
    }
}

// app.module.ts használat:

```

```
// consumer.apply(LoggerMiddleware).forRoutes('*');
```

Pipes (ValidationPipe példa)

```
// src/users/dto/create-user.dto.ts
import { IsString, IsEmail, IsOptional, IsInt } from 'class-validator';

export class CreateUserDto {
  @IsString()
  name: string;

  @IsEmail()
  email: string;

  @IsOptional()
  @IsInt()
  age?: number;
}

// main.ts
import { ValidationPipe } from '@nestjs/common';
app.useGlobalPipes(new ValidationPipe({ transform: true, whitelist: true }));
```

Guards (Role példa)

```
// src/auth/roles.decorator.ts
import { SetMetadata } from '@nestjs/common';
export const Roles = (...roles: string[]) => SetMetadata('roles', roles);

// src/auth/roles.guard.ts
import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}
  canActivate(context: ExecutionContext): boolean {
    const required = this.reflector.get<string[]>('roles', context.getHandler()) || [];
    if (!required.length) return true;
    const req = context.switchToHttp().getRequest();
    const user = req.user;
    return user && required.some(r => user.roles?.includes(r));
  }
}

// használat controlleren
```

```
// @UseGuards(RolesGuard)
// @Roles('admin')
// @Get('admin')
// adminOnly() { return 'secret'; }
```

Interceptor (logging) példa

```
// src/common/interceptors/logging.interceptor.ts
import {
  Injectable,
  NestInterceptor,
  ExecutionContext,
  CallHandler,
} from '@nestjs/common';
import { Observable, tap } from 'rxjs';

@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const now = Date.now();
    return next.handle().pipe(
      tap(() => console.log(`${context.switchToHttp().getRequest().url} - ${Date.now() - now}`)),
    );
  }
}
```

Exception filter példa

```
// src/common/filters/http-exception.filter.ts
import { ExceptionFilter, Catch, ArgumentsHost, HttpException } from '@nestjs/common';
import { Request, Response } from 'express';

@Catch(HttpException)
export class HttpErrorFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const res = ctx.getResponse<Response>();
    const req = ctx.getRequest<Request>();
    const status = exception.getStatus();
    const error = exception.getResponse();
    res.status(status).json({
      statusCode: status,
      timestamp: new Date().toISOString(),
      path: req.url,
      error,
    });
  }
}
```

```
}  
}
```

Kiterjesztett magyarázatok (decoratorok, lifecycle, middleware, pipes, guards, interceptors, routing, renderelés, stb.)

Ez a kiterjesztett fejezet részletesen bemutatja a NestJS fő építőköveit: annotációk/dekorátorok (mire valók és hogyan használjuk őket), provider-ek és scope-ok, middleware-ek, pipes, guards, interceptors, exception filterek, routing és renderelés. Gyakorlati kódrészletekkel illusztrálom a fogalmakat.

Tartalom 1. Rövid áttekintés a request lifecycle-ról 2. Dekorátorok és param-dekorátorok (annotációk) — részletesen 3. Modulok és provider-ek (szolgáltatások) — scope, factory, useClass/useValue 4. Middleware — hol fut, mikor érdemes használni, példák 5. Pipes — transzformációk és validációk, beállítások és példák 6. Guards — jogosultság ellenőrzés, használat, példák 7. Interceptors — response transform, caching, timeout, stream kezelés 8. Exception Filters — hibakezelés, HttpException, custom filter 9. Routing — route felépítés, prefix, versioning, paraméterek, precedence 10. Renderelés és templating — @Render, view engine, static assets 11. Egyéb gyakori annotációk és tippek (file upload, session, headers, response control) 12. GYIK / Best practices

1) Rövid áttekintés a request lifecycle-ról

Amikor egy HTTP kérés érkezik Nest alkalmazásba, a feldolgozás általános sorozata (egyszerűsítve):

1. Express/Fastify bejárja a middleware láncot (global vagy route-specifikus middleware).
2. Guard-ok lefutnak (döntés: engedélyezett-e a végrehajtás).
3. Pipes futnak a paramétereken (transzformáció / validáció).
4. Interceptor-ok “elő” logikája lefut (pl. timer, request inject).
5. Controller -> Handler fut (service-eket hívja).
6. Interceptor-ok “utó” logikája lefut (válasz transzformálás / cache / mapping).
7. Exception filter fogja a dobott hibákat és szabályozott választ ad.

Fontos: middleware a legalsó szinten van (először futnak), a guards és pipes a handler előtt, az interceptors a handler körül vannak (körbefogják), az exception filterek a hibákat fogják meg.

2) Dekorátorok és param-dekorátorok (annotációk) — részletesen

A dekorátorok metaadatok felvitelére szolgálnak TypeScript-ben. NestJS nagyon sokfelé használja őket.

Főkategóriák: - Strukturális dekorátorok: @Module, @Controller, @Injectable — komponensek regisztrálása a DI konténerbe. - Route és HTTP dekorátorok: @Get, @Post, @Put, @Delete, @Patch, @Options, @Head, @All. - Param-dekorátorok: @Body, @Param, @Query, @Headers, @Req, @Res, @Session, @UploadedFile, @UploadedFiles. - Behavior / cross-cutting dekorátorok: @UseGuards, @UseInterceptors, @UsePipes, @UseFilters, @SetMetadata, @Public, @Roles (custom). - Egyedi/developer dekorátorok: createParamDecorator, SetMetadata (pl. @Roles).

Részletek és példa mindegyikre

@Module - Mire jó: deklarálja a modul szintű konfigurációt: controllers, providers, imports, exports, global. - Példa: `typescript @Module({ imports: [UsersModule, AuthModule], controllers: [AppController], providers: [AppService], exports: [], }) export class AppModule {}` - Megjegyzés: @Global() annotációval a modul globálissá tehető — exportált provider-ek minden modulban elérhetők.

@Controller - Mire jó: HTTP végpont csoportosítás (route prefix). - Példa: `typescript @Controller('cats') export class CatsController {}` - Paraméter: prefix string (pl. 'users'), vagy több dekorátorral változtatva.

@CRUD route dekorátorok: @Get, @Post, @Put, @Patch, @Delete - Mire jó: HTTP verbhez kötött handler deklarálása, opcionális útvonal paraméterrel. - Példa: `typescript @Get(':id') findOne(@Param('id') id: string) { return this.service.findOne(id); }`

Param-dekorátorok (mire jó és opciók) - @Param('id') — path params; lehetetlen esetekben @Param() visszaadja az összes parametert. - @Query('q') — querystring paraméterek. - @Body() — request body (általában DTO-val). - @Headers('authorization') — egyedi header. - @Req() / @Res() — alacsony szintű express/fastify request/response, használatukkal elveszíted a framework agnosztikusságot (nem ajánlott általánosan). - @Session() — session objektum (ha session middleware be van kapcsolva). - @UploadedFile() / @UploadedFiles() — file uploadok Multer adapterrel. - Példa: `typescript @Post(':id/upload') upload(@Param('id') id: string, @UploadedFile() file: Express.Multer.File) { ... }`

@Render(template) - Mire jó: visszaad egy nézet renderelt HTML-t a megadott template-fájjal (pl. pug, ejs). - Példa: `typescript @Get() @Render('index') index() { return { title: 'Hello' }; // template változók }`

@HttpCode, @Header, @Redirect - @HttpCode(204) módosítja a válasz HTTP státuszkódját. - @Header('Cache-Control', 'none') beállíthat fejléceket. - @Redirect('/login', 302) átirányítja a kérést.

@Injectable() - Mire jó: jelzi, hogy az osztály provider-ként injektálható. - Példa:
`typescript @Injectable() export class CatsService { ... }`

@Scope() - Alapértelmezett: singleton. - Lehetőségek: Scope.DEFAULT (singleton), Scope.REQUEST, Scope.TRANSIENT. - Request scoped provider-ek per-request példányt adnak (több memóriát, de hasznos request-specifikus state-hez).

@Catch() - Exception filter regisztrálására szolgál. - Példa: @Catch(HttpException)
`export class HttpErrorFilter implements ExceptionFilter { ... }`

@UseGuards, @UseInterceptors, @UsePipes, @UseFilters - Ezek dekorátorok osztályra vagy handler-re helyezhetők. Például @UseGuards(AuthGuard) a controller vagy a route előtt ellenőrzi a hozzáférést.

@SetMetadata / createParamDecorator - createParamDecorator: egyedi param-dekorátor létrehozása (pl. @CurrentUser). - SetMetadata: kulcs-érték metaadatokot állít be, amelyeket később Reflector segítségével olvashatsz (pl. @Roles).

Példa saját dekorátorra:

```
export const CurrentUser = createParamDecorator((data, ctx: ExecutionContext) => {
  const req = ctx.switchToHttp().getRequest();
  return req.user;
});
```

3) Modulok és provider-ek (szolgáltatások)

Provider = - Bármilyen, amit a DI konténer kezel: osztályok (@Injectable), value-k, factory-k. - Regisztrálás: providers: [CatsService] a modulban.

Provider konfigurációk: - useClass: saját class implementáció - useValue: konstans objektum - useFactory: factory függvény (lehet async) - provide: a token (osztály vagy string/symbol) amelyre hivatkozunk

Példa useFactory:

```
{
  provide: 'MY_CLIENT',
  useFactory: async () => {
    const client = await createClientAsync();
    return client;
  }
}
```

Imports / Exports: - Ha egy modul exportál egy provider-t, más modulok, amik importálják ezt a modult, injektálhatják a provider-t. - Példa: DatabaseModule exportálja a 'DATABASE_CONNECTION' token-t, UsersModule importálja a DatabaseModule, így hozzáfér.

Provider scope: - Singleton (alapértelmezett): egész alkalmazásban egyetlen példány. - Request: minden bejövő kérésnél új példány. - Transient: minden injektáláskor új példány.

Lifecycle hooks: - OnModuleInit, OnModuleDestroy, OnApplicationBootstrap, OnModuleDestroy, BeforeApplicationShutdown — implementálhatók és használhatók pl. kapcsolódás/lezárás kezelésére.

Async providers tipikusan:

```
{
  provide: 'DATABASE_CONNECTION',
  useFactory: async () => {
    return await createDbConnection();
  },
}
```

4) Middleware

Middleware-ek Express vagy Fastify rétegben futnak, közvetlenül a kérés feldolgozása előtt. Alkalmask: - Logolás, request body előfeldolgozás (mielőtt Nest feldolgozná). - Kliensoldali SSÓ kezelés, rate-limit, cookie parsing, session kezelő. - Ne használj benne DI-t (bár lehetséges) — inkább szolgáltatásokat injektálj, ha kell.

Két típus: - Funkcionális middleware (function signature: (req,res,next) => void) - Osztály alapú (implements NestMiddleware)

Példa:

```
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    console.log(`${req.method} ${req.url}`);
    next();
  }
}

// AppModule-ban:
configure(consumer: MiddlewareConsumer) {
  consumer.apply(LoggerMiddleware).forRoutes({ path: '*', method: RequestMethod.ALL });
}
```

Rendelkezésre állás: - global (app.use) vagy modul/route-specifikus (consumer.apply(...).forRoutes(...)).

Fontos: - Middleware pipeline sorrendje számít. - Middleware nem rendelkezik automatikus kivételkezeléssel Nest szinten — ha hibát dobsz express módon, azt a Nest is kezeli, de figyelj az aszinkron hibákra.

5) Pipes — transzformációk és validációk

Pipes felelősek a bejövő adatok transzformálásáért és validálásáért, tipikusan controller paramétereken futnak.

Beépített pipe-ok: - ValidationPipe (class-validator + class-transformer) - ParseIntPipe, ParseBoolPipe, ParseUUIDPipe - DefaultValuePipe

ValidationPipe opciók: - transform: true — automatikus típus-transzformáció (pl. '1' -> 1) - whitelist: true — eltávolít minden nem-deklarált mezőt a DTO-ból - forbidNonWhitelisted: true — hibát dob, ha nem engedélyezett mező van - transformOptions: { enableImplicitConversion: true } — implicit típuskonverzió

Példa DTO-val:

```
export class CreateUserDto {
  @IsString()
  name: string;

  @IsEmail()
  email: string;
}

// main.ts
app.useGlobalPipes(new ValidationPipe({ transform: true, whitelist: true }));
```

Custom pipe:

```
@Injectable()
export class ParseIntPipe implements PipeTransform {
  transform(value: any) {
    const val = parseInt(value, 10);
    if (isNaN(val)) throw new BadRequestException('Invalid id');
    return val;
  }
}
```

Mikor használj pipe-ot: - Minden input validálásra és átalakításra. - DTO-k kezelésére a controller határán.

6) Guards — jogosultság ellenőrzés

Guards döntenek arról, hogy a kérés folytatható-e. Alkalmazhatók: - Autentikációs logika (AuthGuard) - Role/permission ellenőrzés - Rate limit szabályok (bár erre inkább dedicated csomag használatos)

Implementáció: - CanActivate interfész implementálása, true/false visszaadása vagy Promise/Observable.

Példa egyszerű RolesGuard:

```
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const roles = this.reflector.get<string[]>('roles', context.getHandler()) || [];
    if (!roles.length) return true;
    const req = context.switchToHttp().getRequest();
    const user = req.user;
    return user && roles.some(r => user.roles?.includes(r));
  }
}
```

AuthGuard (Passport): - @UseGuards(AuthGuard('jwt')) — Passport stratégia használata. - Passport stratégia validálja a token, majd `request.user`-t beállítja.

Guard vs Pipe vs Interceptor: - Guard: boolean döntés, hogy a handler fut-e. - Pipe: adat transzformáció / validáció. - Interceptor: pre-/post-processing (válasz transform, időmérés).

7) Interceptors — mit csinálnak, mikor használd

Interceptors a handler körül futnak, lehetőséget adnak: - Response transzformáció (pl. standardizált válaszforma) - Caching - Timeout kezelés - Logging és időmérés - Binding extra információkhoz (pl. request id) - Manipulálás stream-ekkel (file streaming, large payloads)

Az Interceptor interfész:

```
@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const now = Date.now();
    return next.handle().pipe(
      tap(() => console.log(`Execution time: ${Date.now() - now}ms`)),
    );
  }
}
```

```

    );
  }
}

```

Használat: - `@UseInterceptors(LoggingInterceptor)` vagy globálisan `app.useGlobalInterceptors(new LoggingInterceptor())`.

Response mapping példa (`ClassSerializerInterceptor` használata): - Class-transformer annotációkkal (`@Expose`, `@Exclude`) befolyásolhatod, hogyan nézzen ki a válasz.

Timeout interceptor: - RxJS `timeout` operátorral megszakíthatod a hosszú futásokat és 408/504 típusú hibát adhatsz vissza.

8) Exception Filters — hibakezelés

Exception filter-ek célja a hibák központosított kezelése és a válasz testreszabása.

- Alapvető `HttpException` a Nest-ből (lehetővé teszi a státuskódot és response body-t). - Egyedi filter: `typescript @Catch(HttpException) export class HttpErrorFilter implements ExceptionFilter { catch(exception: HttpException, host: ArgumentsHost) { const ctx = host.switchToHttp(); const response = ctx.getResponse<Response>(); const request = ctx.getRequest<Request>(); const status = exception.getStatus(); response.status(status).json({ statusCode: status, timestamp: new Date().toISOString(), path: request.url, message: exception.message, }); } } - Globális filter regisztráció: app.useGlobalFilters(new HttpErrorFilter());`

Tanács: - Csak a kívánt hibákat csapd le (pl. összetett logika esetén külön filterek, nem mindent egybe).

9) Routing — hogyan épül fel, hogyan működik a routing

Controller/Route alapú routing: - Controller prefix + route dekorátor -> teljes útvonal: `/prefix/route` - Példa: `@Controller('users') + @Get(':id') => GET /users/:id`

Global prefix: - `app.setGlobalPrefix('api');` — minden route elé `“api/”` kerül.

Parameter binding: - Route paraméterek: `@Param('id')`, `@Query('q')`, `@Body()`
 DTO - Több paraméter: `@Param()` visszaadja objektumot.

Route precedence: - Egyező statikus útvonalak (pl. /users/create) előrébb állnak, mint dinamikus paramok (/id). Ha ugyanaz a path, a korábbi regisztráció sorrendje számít.

Versioning: - Nest v6+ támogat route versioning-et. Szabályok: URI versioning (pl. /v1/users), Header-based, Media-type. - Aktiválás: `app.enableVersioning({ type: VersioningType.URI });`

Wildcard és fallback: - `@All('*')` vagy egy dedikált ApplicationController fallback route az SPA-k számára (index.html kiszolgálásához).

Route-level middleware: - `consumer.apply(...).forRoutes(UsersController)` vagy `.forRoutes({ path: 'users', method: RequestMethod.POST })`

10) Renderelés és templating

A Nest Express adapterrel támogatja a templating motorokat (Pug, EJS, Handlebars,...).

Beállítás:

```
// main.ts (Express)
app.setBaseViewsDir(join(__dirname, '..', 'views'));
app.setViewEngine('pug');
```

@Render usage:

```
@Get()
@Render('index')
root() {
  return { title: 'Oldal címe' };
}
```

Alternatív: használhatod az express `res.render()`-t `@Res()` paraméteren keresztül, de ekkor a kontroler elveszti az agnosztikusságát (nem framework-agnosztikus).

Static assets: - `app.useStaticAssets(join(__dirname, '..', 'public'))`; — statikus fájlok kiszolgálása.

Streaming és nagy válaszok: - Használhatod a stream-et `@Res()` segítségével (például nagy fájlok letöltéséhez), vagy az Observable/stream return típusát.

11) Egyéb gyakori annotációk és tippek

@Inject(token) - explicit injection token használata, ha a provider neve string vagy symbol.

@Optional() - jelzi, hogy a provider injektálása opcionális (ha nem létezik, null).

@Global() - globális modul, exportált provider-ek mindenhol elérhetőek.

File Upload - platform-express + multer: `typescript @Post('upload') @UseInterceptors(FileInterceptor('file')) upload(@UploadedFile() file: Express.Multer.File) { ... }`

Multipart több fájl: - @UseInterceptors(FilesInterceptor('files', 10))

Sessions & cookies - express-session middleware beállítása a main.ts-ben (app.use(session(...)))

@SerializeOptions / ClassSerializerInterceptor - Válaszok formázására, DTO-k serializálására.

@UsePipes/Guards/Interceptors helyi használat: - A dekorátorok használhatók controller szinten (minden route), vagy handler szinten (egy route).

Reflexió és Reflector - A Reflector osztály segítségével olvashatók a SetMetadata-szal beállított metaadatok (pl. @Roles).

12) GYIK / Best practices

- Hol validáljunk?
 - Validálás két helyen: DTO + ValidationPipe. DTO-k legyenek tiszták, a pipe legyen globális.
- Mikor használjunk middleware-t vs guard-et?
 - Middleware: alacsony szintű, framework-specifikus műveletek (pl. cors, body parsing, logger).
 - Guard: autorizációs logika, amely a Nest ExecutionContext-hez kapcsolódik.
- Mikor használjunk interceptor-t vs filter-t?
 - Interceptor: ha a válasz formátumát kell módosítani, vagy timeout/caching/logging kell.
 - Filter: ha hibákat kell szabályozott módon kezelni.
- Scope-ok — mikor Request scoped?
 - Ha a provider tartalmaz request-specifikus state-et (pl. audit log, request id). Kerüld, ha nem szükséges, mert perf. költséges.
- Templating vs API:
 - Ha API-first (REST/GraphQL), ne keverd a view renderelést; külön endpointok javasoltak.
- DI tokenek:

- Használj `Symbol()`-t vagy readonly `const` stringeket, hogy elkerüld a névütközéseket.
- Tesztelés:
 - Mockold a provider-eket a `Test.createTestingModule({ providers: [{ provide: UsersService, useValue: mock }] })` mintán.
 - Integrációs tesztek: használd a `Test.createTestingModule` és a `supertest` könyvtárat.

Melléklet — példák: összehasonlítás Guard / Pipe / Interceptor / Middleware

- Middleware: loggolás, body parsing
 - futás: `express` -> `middleware` -> `nest`
- Guard: autentikáció / roles
 - futás: `middleware` után, `pipeline` előtt
- Pipe: DTO validáció / `parse` int
 - futás: param-ok betöltésekor, a handler előtt
- Interceptor: response mapping / cache / timeout
 - futás: handler körül, before and after

Ha szeretnéd, ezt a kiterjesztett anyagot beillesztem a `repo-dba` (pl. `docs/NestJS-guide-extended.md`), illetve előkészítek hozzá egy `pandoc`-szabályt a PDF generálásához. Mondd meg kérlek: - Illesszem be a korábbi `NestJS Markdownba` (helyettesítve), vagy hozzak létre külön fájlt: `docs/NestJS-guide-extended.md`? - Kíváncsi, hogy generáljak egy frissített `GitHub Actions workflow`-ot, amely mindkét `Markdown` fájlból PDF-et készít?

Szívesen bontom tovább a szekciókat külön fájlokra (például `decorators.md`, `lifecycle.md`) ha dokumentációs oldalakat tervezel.